# GESP-8 质量抽检模拟试卷（含答案与解析）

考试时间：60 分钟　满分：100 分　生成时间：2026/05/07 08:39:56

姓名：_____ 班级：_____ 得分：_____

请在答题区域作答；单项选择题、阅读程序题和完善程序题均为单选。除特别说明外，程序均按 C++17 标准理解。

抽样来源：papers/2026 已入库题库文件仓库；蓝图版本：config/blueprint.ts v1；抽样种子：20260507034045

## 一、单项选择题（共 15 题，每题 2 分，共 30 分）

1. （2 分）对序列 {4, 2, 7, 1, 5} 进行升序插入排序，第三趟排序后序列变为？

A. {1, 2, 7, 4, 5}

B. {2, 4, 7, 1, 5}

C. {1, 2, 4, 7, 5}

D. {1, 2, 4, 5, 7}

**答案：C**

**解析：**

插入排序过程：初始 {4,2,7,1,5}；第一趟：插入 2 -> {2,4,7,1,5}；第二趟：插入 7 -> {2,4,7,1,5}；第三趟：插入 1 -> {1,2,4,7,5}。

2. （2 分）背包问题：有n=4件物品，重量w={2,3,4,5}，价值v={3,4,5,6}，背包容量C=8。使用动态规划求解0-1背包问题，获得的最大价值是？

A. 8

B. 9

C. 10

D. 11

**答案：C**

**解析：**

物品：1(2,3), 2(3,4), 3(4,5), 4(5,6)。容量8。选择物品1,2,3：总重2+3+4=9>8不行；物品1,2,4：重2+3+5=10>8；物品1,3,4：重2+4+5=11>8；物品2,3,4：重3+4+5=12>8；物品1,4：重7，价值9；物品2,3：重7，价值9；物品3,4：重9>8；物品1,2,3的重量超了，排除。检查组合：选2,4：重8，价值10；选1,3,?不行；选1,2：重5价值7；选3,? 等等。最大为物品2和4组合，价值10。或者动态规划表：dp[i][j]表示前i个物品容量j的最大价值。计算得dp[4][8]=10。

3. （2 分）以下C++程序段用于计算斐波那契数列的第 n 项（n>=1），但缺少一行代码。请选择填入空缺处（??）的正确语句。

```
int fib(int n) {
 if (n == 1 || n == 2) return 1;
 else return ??;
}
```

A. fib(n-1) + fib(n-2)

B. fib(n+1) + fib(n+2)

C. fib(n-2) + fib(n-3)

D. fib(n) + fib(n-1)

**答案：A**

**解析：**

斐波那契数列定义：F(1)=F(2)=1，对于 n>2，F(n)=F(n-1)+F(n-2)。所以空缺处应填入 fib(n-1) + fib(n-2)。其他选项均不符合递推关系。

4. （2 分）在C++中，以下代码的输出是什么？

```
#include <iostream>
```

```
using namespace std;
int main() {
 int a = 5, b = 3;
 bool c = (a > b) && (b++ > 2);
 cout << b << endl;
 return 0;
}
```

A. 3

B. 4

C. 5

D. 编译错误

**答案：B**

**解析：**

表达式(a > b) && (b++ > 2)中，a>b为5>3为true，然后计算b++>2，b当前为3，3>2为true，之后b自增为4。由于短路求值，当第一个条件为true时，第二个条件会执行，所以b++被计算，b变为4。输出b时，b的值是4。因此正确答案是B。

5. （2 分）在一棵完全二叉树中，节点总数为 15。如果按照“根节点深度为1”的定义，那么这棵树的深度是多少?

A. 3

B. 4

C. 5

D. 6

**答案：B**

**解析：**

完全二叉树的深度（depth）h 与节点数 n 满足关系：2^{h-1} ≤ n < 2^h。
对于 n=15：
- 2^{4-1} = 8 ≤ 15
- 2^4 = 16 > 15
因此，深度 h = 4。这与“根节点深度为1”的定义一致。
注意：通常“高度”定义为从根到叶的最长路径上的边数，此时单节点树高度为0，15节点的完全二叉树高度为3。但本题明确采用“根节点深度为1”的定义，因此答案为4。

6. （2 分）对于一个完全二叉树，若节点总数为15，则该树的深度（根节点深度为0）是多少?

A. 3

B. 4

C. 5

D. 6

**答案：A**

**解析：**

完全二叉树深度d与节点数N的关系：2^(d+1) - 1 ≥ N > 2^d - 1。代入N=15：2^4 - 1 = 15，因此深度d = 3（因为深度从0开始计数）。或者直接计算：第0层1个节点，第1层2个，第2层4个，第3层8个，总和为15，所以深度为3。

7. （2 分）设有向图G采用邻接表存储，顶点个数为n，边数为e。删除图中所有以顶点v为起点的边的时间复杂度为（假设查找v的邻接表头结点时间为O(1)，仅需将边从链表中移除，不要求释放节点内存）？

A. O(1)

B. O(out-degree(v))

C. O(n)

D. O(e)

**答案：B**

**解析：**

在邻接表中，以顶点v为起点的所有边存储在v对应的链表中。要删除这些边，必须遍历v的整个邻接链表，将每个边节点从链表中移除。这个过程的时间复杂度取决于v的出度，即O(out-degree(v))。最坏情况下，v的出度为n-1，因此时间复杂度为O(n)。选项A错误，因为无法通过简单地将头指针置空来满足“将边从链表中移除”的要求（这仅是丢弃引用，而非从链表中移除节点）。

8. （2 分）已知一个完全二叉树有 100 个节点，其根节点的深度为 1（即根节点在第 1 层）。则该树中度为 0 的节点（叶子节点）的数量是多少？

A. 50

B. 51

C. 49

D. 52

**答案：A**

**解析：**

对于完全二叉树，节点总数 n = 100。设叶子节点数为 n0，度为 1 的节点数为 n1，度为 2 的节点数为 n2。有性质：n0 = n2 + 1。完全二叉树中，度为 1 的节点数 n1 只能是 0 或 1（因为节点按层序编号，只有最后一层可能不满，且如果不满，最后一个父节点可能只有一个孩子）。由 n = n0 + n1 + n2 = 2n0 - 1 + n1，得 2n0 = n + 1 - n1 = 101 - n1。因为 n0 是整数，所以 101 - n1 必须是偶数。n1 只能取 0 或 1。若 n1=0，则 2n0=101，n0 不是整数，矛盾。若 n1=1，则 2n0=100，n0=50。因此叶子节点数为 50。所以答案是 A。

9. （2 分）以下关于二进制数的描述，正确的是：

A. 二进制数 1011 转换为十进制是 12

B. 十进制数 13 转换为二进制是 1101

C. 二进制数 1100 和 1010 的按位与结果是 1010

D. 二进制数 1001 逻辑右移一位后得到 1000

**答案：B**

**解析：**

A. 二进制 1011 转换为十进制：$1\times2^3 + 0\times2^2 + 1\times2^1 + 1\times2^0 = 8 + 0 + 2 + 1 = 11$，因此 A 错误。

B. 十进制 13 转换为二进制：13 ÷ 2 = 6 余 1，6 ÷ 2 = 3 余 0，3 ÷ 2 = 1 余 1，1 ÷ 2 = 0 余 1，从下往上读取余数得到 1101，因此 B 正确。

C. 二进制 1100 和 1010 的按位与运算：逐位进行与操作（1&1=1，1&0=0，0&1=0，0&0=0），结果为 1000，因此 C 错误。

D. 二进制数 1001 逻辑右移一位：右移时左侧补 0，变为 0100，即 100，因此 D 错误。

10. （2 分）在C++中，执行以下代码片段后，变量 `x` 的值是多少？

```
int a = 5, b = 3, x = 0;
if (a > b) {
    if (b > 2) x = 1;
    else x = 2;
} else {
    x = 3;
}
```

A. 0

B. 1

C. 2

D. 3

**答案：B**

**解析：**

首先，外层条件 `a > b` 为真（5 > 3），因此进入外层if块。内层条件 `b > 2` 也为真（3 > 2），因此执行 `x = 1`。所以最终 `x` 的值为1。

11. （2 分）长度为 10 的二进制串中，恰好有 3 个字符是 1，这样的二进制串一共有多少个？

A. 30
B. 60
C. 90
D. 120

**答案：D**

**解析：**

从 10 个位置中选出 3 个位置放 1，其余位置放 0，方案数为组合数 C(10,3)=120。

12. （2 分）一个 8 节点的完全二叉树用数组存储（下标从 1 开始），层序遍历的顺序恰好是 1, 2, 3, 4, 5, 6, 7, 8。问：进行中序遍历时，访问到的第 5 个节点的值是多少？

A. 5
B. 6
C. 7
D. 8

**答案：B**

**解析：**

根据完全二叉树的数组存储规则（下标从1开始），层序遍历顺序即为数组下标顺序。因此，根节点下标为1，值为1；其左孩子下标为2，值为2；右孩子下标为3，值为3；下标4的节点值为4，是节点2的左孩子；下标5的节点值为5，是节点2的右孩子；下标6的节点值为6，是节点3的左孩子；下标7的节点值为7，是节点3的右孩子。中序遍历（左子树-根-右子树）的完整访问序列为：4（左子树最左）→ 2（左子树根）→ 5（左子树右）→ 1（整棵树的根）→ 6（右子树左）→ 3（右子树根）→ 7（右子树右）。因此，第5个访问到的节点值是6。

13. （2 分）执行以下C++代码后，输出结果是什么？

```
int x = 10;
if (x > 5) {
    x = x - 5;
} else if (x > 8) {
    x = x - 3;
} else {
    x = 0;
}
std::cout << x;
```

A. 5
B. 7
C. 8
D. 0

**答案：A**

**解析：**

`x`初始为10，第一个条件`x > 5`为真，执行`x = x - 5`，`x`变为5。由于`if-else if-else`结构，一旦某个条件满足，后续的`else if`和`else`分支将被跳过。因此，最终`x`为5，输出5。

14. （2 分）有以下 C++ 程序：

```
#include <iostream>
using namespace std;
int func(int &x, int y) {
    x = x + y;
    return x - y;
}
int main() {
    int a = 10, b = 3;
    int c = func(a, b);
    cout << a << " " << b << " " << c << endl;
    return 0;
}
```

运行后输出是什么？

A. 10 3 7
B. 13 3 10
C. 7 3 10
D. 13 3 7

**答案：B**

**解析：**

func 中 x 是 a 的引用，y 是 b 的副本。x = x + y 使 a 变为 10+3=13，返回 x - y = 13-3=10，c=10。输出 a=13, b=3, c=10。

15. （2 分）以下代码段执行后，输出是什么？

```
#include <iostream>
#include <string>
using namespace std;
int main() {
    string s = "abc";
    string& r = s;
    r += "def";
    string t = move(s);
    cout << s.size() << " " << t << endl;
    return 0;
}
```

A. 6 abcdef
B. 0 abcdef
C. 6
D. 0 abc

**答案：B**

**解析：**

首先s初始化为"abc"，r是s的引用，通过r追加"def"，此时s变为"abcdef"，长度为6。然后执行string t = move(s);，move将s转换为右值引用，移动构造t，将s的资源转移给t。在移动后，s处于有效但未指定的状态。根据C++标准，对于std::string，移动后原字符串通常为空字符串，size()返回0。因此输出s.size()为0，t为"abcdef"。选项B正确。

## 二、阅读程序题（共 3 题，每题 5 小题，共 40 分；小题分值见题内标注）

16. （本题 14 分）以下C++程序使用前缀和算法处理一个整数数组。请阅读程序并回答问题。

```
#include <iostream>
#include <vector>
using namespace std;
int main() {
    vector<int> arr = {2, 5, 1, 4, 3, 2, 6, 1};
    int n = arr.size();
    vector<int> pre(n + 1, 0);
    for (int i = 0; i < n; ++i) {
        pre[i + 1] = pre[i] + arr[i];
    }
    int q1 = 2, q2 = 5;
    int sum1 = pre[q2 + 1] - pre[q1];
    int q3 = 1, q4 = 6;
    int sum2 = pre[q4 + 1] - pre[q3];
    int total = sum1 + sum2;
    cout << "sum1:" << sum1 << endl;
    cout << "sum2:" << sum2 << endl;
    cout << "total:" << total << endl;
    return 0;
}
```

16.1 （3 分）程序运行后，输出行"sum1:"后面打印的数值是多少？

A. 10

B. 11

D. 15

**答案：A**

16.2 （3 分）输出行"sum2:"后面打印的数值是多少？

A. 21

C. 23

D. 24

**答案：A**

16.3 （3 分）输出行"total:"后面打印的数值是多少？

A. 29

B. 30

C. 31

D. 32

**答案：C**

16.4 （3 分）前缀和数组pre在索引4处的值是多少？

A. 10

B. 12

C. 14

D. 15

**答案：B**

16.5 （2 分）程序中变量n的值是多少？

A. 6

B. 7

C. 8

D. 9

**答案：C**

**解析：**

1. 计算前缀和数组pre: pre[0]=0, pre[1]=2, pre[2]=7, pre[3]=8, pre[4]=12, pre[5]=15, pre[6]=17, pre[7]=23, pre[8]=24。sum1 = pre[q2+1] - pre[q1] = pre[6] - pre[2] = 17 - 7 = 10。程序打印"sum1:10"，对应选项A。
2. sum2 = pre[q4+1] - pre[q3] = pre[7] - pre[1] = 23 - 2 = 21。程序打印"sum2:21"，对应选项A。
3. total = sum1 + sum2 = 10 + 21 = 31。程序打印"total:31"，对应选项C。
4. pre[4]是前4个元素和：arr[0]+arr[1]+arr[2]+arr[3] = 2+5+1+4 = 12。所以pre[4]=12，对应选项B。
5. arr有8个元素，n = arr.size() = 8。所以n=8，对应选项C。

17. （本题 13 分）阅读以下C++17程序，回答问题。

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    int n = 6;
    vector<int> a = {5, 2, 8, 3, 1, 4};
    vector<int> b(n, 0);
    int cnt = 0;
```

```
    for (int i = 0; i < n; ++i) {
        for (int j = i + 1; j < n; ++j) {
            if (a[i] > a[j]) {
                b[i]++;
            } else {
                b[j]++;
            }
        }
    }
    for (int i = 0; i < n; ++i) {
        if (b[i] == n - 1 - i) {
            cnt++;
        }
    }
    cout << "cnt=" << cnt << endl;
    vector<int> c(n);
    for (int i = 0; i < n; ++i) {
        c[b[i]] = a[i];
    }
    for (int i = 0; i < n; ++i) {
        cout << c[i];
        if (i < n - 1) cout << " ";
    }
    cout << endl;
    return 0;
}
```

17.1 （3 分）程序执行后，第一行输出的 `cnt` 值是多少？

A. 1

B. 2

C. 3

D. 4

**答案：A**

17.2 （3 分）程序执行后，第二行输出的序列中，第一个元素是什么？

A. 1

B. 2

C. 3

D. 4

**答案：A**

17.3 （3 分）程序执行后，第二行输出的序列中，最后一个元素是什么？

A. 5

B. 6

C. 7

D. 8

**答案：D**

17.4 （2 分）程序执行后，第二行输出的序列是？

A. 1 2 3 4 5 6

B. 1 2 3 4 5 8

C. 1 2 3 4 6 8

D. 1 2 3 5 6 8

**答案：B**

17.5 （2 分）程序执行后，b数组中哪个索引的值等于0？

A. 索引0

B. 索引1
C. 索引4
D. 索引5

**答案：C**

**解析：**

1. 数组a = {5,2,8,3,1,4}。计算b数组：通过两层循环，比较每一对(i,j)，如果a[i] > a[j]则b[i]++，否则b[j]++。手动模拟得到b = [4,1,5,2,0,3]。然后检查b[i] == n-1-i（即5-i）：i=3时b[3]=2等于5-3=2，其他都不满足，所以cnt=1。因此输出cnt=1，选项A正确。
2. 根据计算，b = [4,1,5,2,0,3]。构造c数组：c[b[i]] = a[i]，得到c[4]=a[0]=5, c[1]=a[1]=2, c[5]=a[2]=8, c[2]=a[3]=3, c[0]=a[4]=1, c[3]=a[5]=4。所以c数组为[1,2,3,4,5,8]，第一个元素是1，选项A正确。
3. c数组输出为1 2 3 4 5 8，最后一个元素是8，选项D正确。
4. c数组输出序列是1 2 3 4 5 8，选项B正确。
5. b数组为[4,1,5,2,0,3]，只有索引4的值为0，选项C正确。

18. （本题 13 分）阅读以下C++程序，根据程序中的初始值回答问题。

```
#include <iostream>
#include <stack>
#include <queue>
using namespace std;
int main() {
    int n = 7;

    stack<int> s;
    queue<int> q;
    for (int i = 0; i < n; ++i) {
        s.push(i % 3);
        q.push(i % 3);
    }
    int cnt = 0;
    while (!s.empty() && !q.empty()) {
        if (s.top() == q.front()) {
            cnt++;
            s.pop();
            q.pop();
        } else {
            s.pop();
            q.pop();
        }
    }
    cout << cnt << endl;
    cout << s.size() << endl;
    cout << q.size() << endl;
    return 0;
}
```

18.1 （3 分）当 n 初值为 7 时，程序第一个输出（cnt）的值是?

A. 2
B. 3
C. 4
D. 5

**答案：B**

18.2 （3 分）当 n 初值为 7 时，程序第二个输出（s.size()）的值是?

A. 0

B. 1

C. 2

D. 3

**答案：A**

18.3 （3 分）当 n 初值为 7 时，程序第三个输出（q.size()）的值是?

A. 0

B. 1

C. 2

D. 3

**答案：A**

18.4 （2 分）当 n 初值为 5 时，程序第一个输出（cnt）的值是?

A. 1

B. 2

C. 3

D. 4

**答案：A**

18.5 （2 分）当 n 初值为 5 时，程序输出结束后，栈s和队列q的大小分别是多少?

A. 0和0

B. 1和0

C. 0和1

D. 1和1

**答案：A**

**解析：**

1. n 初值为 7 时，循环i从0到6，i%3依次得到序列：0,1,2,0,1,2,0。栈s依次压入0,1,2,0,1,2,0（栈顶为最后一个元素0）；队列q依次压入0,1,2,0,1,2,0（队首为第一个元素0）。随后while循环：第一次，s.top()=0, q.front()=0，相等，cnt=1，栈s变为[0,1,2,0,1,2]（顶为2），队列q变为[1,2,0,1,2,0]（首为1）。第二次，s.top()=2, q.front()=1，不等，栈s变为[0,1,2,0,1]（顶为1），队列q变为[2,0,1,2,0]（首为2）。第三次，s.top()=1, q.front()=2，不等，栈s变为[0,1,2,0]（顶为0），队列q变为[0,1,2,0]（首为0）。第四次，s.top()=0, q.front()=0，相等，cnt=2，栈s变为[0,1,2]（顶为2），队列q变为[1,2,0]（首为1）。第五次，s.top()=2, q.front()=1，不等，栈s变为[0,1]（顶为1），队列q变为[2,0]（首为2）。第六次，s.top()=1, q.front()=2，不等，栈s变为[0]（顶为0），队列q变为[0]（首为0）。第七次，s.top()=0, q.front()=0，相等，cnt=3，栈s变为空，队列q变为空。循环结束，cnt=3。
2. 根据上一问的跟踪，while循环结束后栈s变为空，因此s.size()为0。
3. 根据上一问的跟踪，while循环结束后队列q也变为空，因此q.size()为0。
4. 输入n=5时，循环i从0到4，i%3依次得到序列：0,1,2,0,1。栈s依次压入0,1,2,0,1（栈顶为1）；队列q依次压入0,1,2,0,1（队首为0）。随后while循环：第一次，s.top()=1, q.front()=0，不等，栈s变为[0,1,2,0]（顶为0），队列q变为[1,2,0,1]（首为1）。第二次，s.top()=0, q.front()=1，不等，栈s变为[0,1,2]（顶为2），队列q变为[2,0,1]（首为2）。第三次，s.top()=2, q.front()=2，相等，cnt=1，栈s变为[0,1]（顶为1），队列q变为[0,1]（首为0）。第四次，s.top()=1, q.front()=0，不等，栈s变为[0]（顶为0），队列q变为[1]（首为1）。第五次，s.top()=0, q.front()=1，不等，栈s变为空，队列q变为空。循环结束，cnt=1。
5. 根据上一问的跟踪，while循环结束后栈s和队列q都变为空，因此大小均为0。

## 三、完善程序题（共 2 题，共 30 分；各空分值见题内标注）

19. （本题 15 分，共 4 空）给定一个包含 n 个正整数的数组 a，我们需要找出数组中所有满足 i < j 且 a[i] + a[j] 为偶数的无序对 (i, j) 的数量。请补全以下 C++17 程序。

```
#include <iostream>
#include <vector>
```

```
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> a(n);
    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }
    int cnt_even = 0, cnt_odd = 0;
    for (int i = 0; i < n; ++i) {
        if (a[i] % 2 == 0) {
            BLANK1;
        } else {
            BLANK2;
        }
    }
    long long ans = 0;
    ans += BLANK3; // 计算偶数之间的配对数
    ans += BLANK4; // 计算奇数之间的配对数
    cout << ans << endl;
    return 0;
}
```

样例输入：

```
5
2 3 4 5 6
```

样例输出：

```
4
```

空 1（BLANK1，4 分）

A. cnt_even = cnt_even + 1

B. cnt_even++

C. cnt_even += 2

D. ++cnt_even

**答案：B**

空 2（BLANK2，4 分）

A. cnt_odd = cnt_odd + 1

B. cnt_odd++

C. cnt_odd += 2

D. ++cnt_odd

**答案：B**

空 3（BLANK3，4 分）

A. cnt_even * cnt_even

B. cnt_even * (cnt_even - 1) / 2

C. cnt_even * (cnt_even + 1) / 2

D. cnt_even * cnt_even / 2

**答案：B**

空 4（BLANK4，3 分）

A. cnt_odd * cnt_odd

B. cnt_odd * (cnt_odd - 1) / 2

C. cnt_odd * (cnt_odd + 1) / 2

D. cnt_odd * cnt_odd / 2

**答案：B**

**解析：**

1. 选项A和D功能与B相同，但B更简洁，且题目要求只有一个正确选项。选项C错误，会多计数。
2. 与BLANK1类似，选项A和D功能相同，但B更简洁。选项C错误。

3. 计算从cnt_even个偶数中选2个的组合数，公式为C(n,2)=n*(n-1)/2。选项A是平方，选项C和D公式错误。
4. 计算从cnt_odd个奇数中选2个的组合数，公式为C(n,2)=n*(n-1)/2。选项A、C、D公式错误。

**参考完整代码：**

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    int n;
    cin >> n;
    vector<int> a(n);
    for (int i = 0; i < n; ++i) {
        cin >> a[i];
    }
    int cnt_even = 0, cnt_odd = 0;
    for (int i = 0; i < n; ++i) {
        if (a[i] % 2 == 0) {
            cnt_even++;
        } else {
            cnt_odd++;
        }
    }
    long long ans = 0;
    ans += cnt_even * (cnt_even - 1) / 2;
    ans += cnt_odd * (cnt_odd - 1) / 2;
    cout << ans << endl;
    return 0;
}
```

20. （本题 15 分，共 3 空）给定两个正整数 a 和 b，求它们的最小公倍数（LCM）和最大公约数（GCD）之积与 a*b 是否相等。补全程序，输入 a 和 b，输出验证结果（1 表示相等，0 表示不相等）。

```
#include <iostream>
using namespace std;

int gcd(int x, int y) {
    while (y != 0) {
        int temp = y;
        y = BLANK1;
        x = temp;
    }
    return x;
}

int main() {
    int a, b;
    cin >> a >> b;
    int g = gcd(a, b);
    int l = BLANK2;
    int product1 = a * b;
    int product2 = l * g;
    int result = BLANK3;
    cout << result << endl;
    return 0;
}
```

样例输入：

```
12 18
```

样例输出：

```
1
```

空 1（BLANK1，5 分）

A. x % y

B. x / y

C. x - y

D. temp % x

**答案：A**

空 2（BLANK2，5 分）

A. a / g * b

B. a * b / g

C. a * b % g

D. a / g + b / g

**答案：A**

空 3（BLANK3，5 分）

A. product1 == product2

B. product1 != product2

C. product1 >= product2

D. product1 <= product2

**答案：A**

**解析：**

1. 欧几里得算法求最大公约数的核心是辗转相除法，循环中 y 应更新为 x 对 y 取模的结果。选项 B 是除法，会直接得到商而不是余数，破坏算法；选项 C 是减法，虽然也能求 GCD，但效率低且与标准辗转相除法不一致；选项 D 使用 temp % x，但此时 temp 已更新为原来的 y，x 为原来的 x，逻辑错误，无法正确计算余数。
2. LCM(a,b) = a*b / GCD(a,b)，为防止溢出，通常先除后乘，即 (a/g)*b。选项 B 先乘后除，可能先导致 a*b 整数溢出；选项 C 是取模运算，得到的不是 LCM；选项 D 是除法后相加，完全错误。
3. 题目要求验证 LCM*GCD 是否等于 a*b，即判断 product1 和 product2 是否相等。选项 B、C、D 的比较逻辑与题目要求不符，其中选项 B 判断不等，会输出相反结果；选项 C 和 D 是大于等于或小于等于，在相等时虽然可能输出1，但当不相等时逻辑错误，且不能保证正确性（例如当 product1 < product2 时，选项 C 输出0，但 product1 可能等于 product2）。只有选项 A 正确表达相等关系。

**参考完整代码：**

```
#include <iostream>
using namespace std;

int gcd(int x, int y) {
    while (y != 0) {
        int temp = y;
        y = x % y;
        x = temp;
    }
    return x;
}

int main() {
    int a, b;
    cin >> a >> b;
    int g = gcd(a, b);
    int l = a / g * b;
    int product1 = a * b;
    int product2 = l * g;
    int result = product1 == product2;
    cout << result << endl;
    return 0;
}
```